

Lab2: MoE 的向量化计算实验报告

徐健乘 3250105245

1 AI Agent 使用情况

Lab1 中我仅以对话形式向 AI 提问；Lab2 的 AI 介入更深一些。基础向量化（AVX-512 VNNI intrinsics）、AMX tile 配置、OpenMP 并行结构由我自己手写，后期为节约时间，借助 Claude Code 的 Agent 模式帮忙实现优化思路和跑集群验证。我还搭了一个夜间自动验证系统，让 AI 夜间批量跑实验，白天再根据结果决定下一步方向。

实际跑下来的体会是：优化方向主要得自己想。比如把同一专家的多个 token 合并成一次 FFN 调用来复用权重、把 router 从 per-token VNNI 换成 batched AMX GEMM、把 combine 改成单趟流式，这些结构性的改造都是从 VTune 热点数据反推出来的，AI 不太会主动提出这类方向。但只要我给清楚优化假设，AI 能很快把代码实现出来、在集群上跑正确性和 paired benchmark、甚至做汇编审计，这部分比人快得多。

AI 也会帮倒忙。最有代表性的一个例子：batched FFN 的代码实现后，S4 快了 40%，但 S2 莫名其妙慢了 11472%。根因是新增的 330 行 intrinsics 挤占 icache，改变了编译器对小 N 路径的 inline 决策。这种二进制级副作用 AI 完全预测不到，最后还是靠真实集群的 paired benchmark 才发现，再用 `noinline+cold` 隔离解决。另外 AI 静态估算收益经常偏乐观，比如 sigmoid 跳过它估能省 35%，实测只有 2%，因为乱序执行会隐藏非关键路径的延迟。所以判断一个改动值不值，得看它是否在关键路径上，而不是看 AI 估的省了多少计算。

2 实验环境

2.1 集群硬件

2.2 问题规模与评分曲线

四个评测场景的形状 (N, D, H, E, K) 与评分 checkpoint 如下，评分按对 scalar 基线的加速比 p 经非线性曲线映射：

评分曲线在 60 分 checkpoint 前为 0，60-100 之间指数增长，100-120 之间线性递减边际。正确性容差较宽松（per-token 相对 L2 $< 2 \times 10^{-2}$ ，全局 RMSE $< 2 \times 10^{-3}$ ），因为 int8 重量化 $\text{round}(h/s_h)$ 的舍入边界会让一个 int8 值差 ± 1 。

表 1: x86 主线评测环境

组件	规格
计算节点	Intel Xeon Gold 5418Y (Sapphire Rapids)
核/线程	8 物理核 / 16 线程 (-c 16, cgroup 钉单 socket)
向量扩展	AVX-512 VNNI + AMX (INT8/BF16)
编译器	GCC 14, -O3 -march=sapphirerapids -fopenmp (仅 student 目标)
Profiling	VTune (hotspots) + perf stat -topdown

表 2: 四场景形状与评分 checkpoint (加速比)

场景	N	D	H	E	K	60/100/120 分 checkpoint
S1	1	256	128	16	4	14 / 20 / 26
S2	1	1024	512	16	4	8 / 15 / 19
S3	128	256	128	16	4	50 / 100 / 150
S4	1024	512	128	512	2	55 / 120 / 275

3 MoE 前向流程与 W8A8 量化

参考实现是逐 token 串行的标量版本，每个 token 的前向流程为：

1. **路由**：计算亲和度 $s_e = \sigma(w_{\text{router},e} \cdot x_t)$ 。
2. **Top-K**：按 $s_e + \text{bias}_e$ 选最大的 K 个专家 (bias 仅用于选择，不进 gate 值)。
3. **量化**：per-token 对称量化 $x_q = \text{round}(x_t/s_x)$, $s_x = \max |x_t|/127$ 。
4. **SwiGLU FFN** (共享专家 + 每个 Top-K 路由专家)： $h = \text{SiLU}(W_g x_q) \odot (W_u x_q)$, 重量化 h_q , 再 $o = W_d h_q$ 。权重是 per-matrix scale 的 int8, 乘积在 int32 累加后反量化回 fp32。
5. **合并**： $y_t = x_t + o_{\text{shared}} + \sum_e g_e o_e$, 其中 $g_e = s_e / \sum_{j \in S_t} s_j$ 。

基线的主要问题是访存模式差：每个 token 都要完整读一遍它所选专家的全部权重，且标量循环指令总数巨大。prof perf topdown 显示初始框架 S3 的 Retiring 高达 75%、Memory Bound 仅 0.4%、IPC ≈ 4.7 ，流水线没空转，只是在高效执行海量标量指令，每条只完成一次 8-bit 乘加。所以优化方向是提高单条指令完成的工作量（向量化 + 矩阵扩展），以及减少权重重复加载（数据重排 + 权重复用）。

4 优化思路

4.1 整体结构

整个前向计算分三步走：先 router 对每个 token 算 512 个专家的得分选出 Top-K，同时把 token 量化成 int8；再对每个选中的专家跑 SwiGLU FFN (gate、up 投影 + SiLU 激活 + down 投影)；最后把所有专家的输出加权加到残差上，得到最终结果。参考实现这三步都是逐 token 串行、逐元素标量循环，慢在两个地方：一是每条指令只算一次乘加，二是同一个专家的权重被反复读取。

优化就从这两处下手。一是用 AVX-512 VNNI 和 AMX 把标量循环换成向量指令，一条指令算 16 到 1024 个乘加；二是把权重在 preprocess 阶段重排好、把同专家多 token 合并复用、把 combine 改成单趟流式，减少重复访存。另外 S4 的 router 量化到 int8 后会有约 1% 的误差，可能选错专家，需要分两阶段选择来兜底正确性。下面按这几个方向分别讲。

4.2 基本向量化

用 AVX-512 VNNI 和 AMX 两套向量指令替换标量循环。VNNI 的 `dpmulb` 一条指令同时算 16 个 int8 乘法并把结果累加到 int32。AMX 则更快，满 tile 一次算 1024 个乘加。把原来逐元素循环换成这两条指令，算力密度直接提一个量级。

VNNI 端口每 0.5 周期能吃一条指令，但如果后一条要等前一条算完才能开始（数据依赖），端口就空了。把 K 维循环展开 4 倍，让 8 组独立的累加器同时算，VNNI 端口在等数据加载的时候也有活干。

AMX 不是全部适用。AMX 的 tile 一次要喂 16 个 token 才能达到最大效率，如果只有一个 token (S1/S2)，15/16 的算力都浪费了，实测反而比 VNNI 慢。所以按场景分：大 N 的 S4 用 AMX 算 router（一次 16 专家 $\times$ 16 token），小 N 走 VNNI，路由专家始终稀疏也走 VNNI（专家分桶的收益抵不过而外开支）。这个取舍让 S4 的 router 占用从 33% 降到 3.8%。

4.3 减少重复加载

参考实现每个 token 都要把它选中的专家权重从头到尾读一遍。同一个专家被多个 token 选中时，权重被反复从内存读进来又丢掉。如果能读一次权重、给多个 token 复用，访存就省了。这条主线的优化都在解决这个问题。

preprocess 重排权重布局。向量指令要求权重按特定格式排列（VNNI 要 16 行一组、AMX 要 16×64 的 tile），如果每次算的时候临时拼，开销很大。所以在计时前一次性把权重重排好，算的时候直接连续读取，cache 命中率好。

(token, slot) 扁平并行。参考实现先算完一个 token 的共享专家，再串行算 4 个路由专家，16 个线程只有 5 个在干活。把所有 token 的所有专家任务拍平到一个列表

里，让 16 个线程一起领活干，每个任务写自己的输出缓冲，不用抢。S1/S2 从约 2x 提升到约 9x。

batched expert FFN。这是收益最大的一步。S4 里同一个专家平均被 4 个 token 选中，原来每个 token 独立调一次 FFN，同一个专家的权重被读了 4 遍。改成把这 4 个 token 合并到一次调用里，权重只读一遍，在 4 个 token 之间复用。S4 从 87x 提到 151x，S3 从 51x 提到 75x。

这步踩了个坑：加完 batched 代码后 S4 快了 40%，但 S2 莫名其妙慢了 114 倍。S2 根本不跑 batched 代码，慢的原因是新加的 330 行指令把指令缓存挤乱了，编译器对 S2 那条小路径的优化决策也跟着变了。这种问题看源码看不出来，得在真实集群上对比测试才发现。解决办法是把 batched 函数标记为冷代码推到代码段末尾，加个判断只让 S4 走新路径。

combine 流水线融合。原来把所有专家输出加起来要来回读写输出数组好几趟：先写残差 + 共享专家，再逐个把路由专家加进去。改成在寄存器里累加，所有专家用一条流式加法链折进去，最后写一次到内存。读写次数减半，数据在寄存器里保持热。

4.4 AMX router

S4 有 512 个专家，router 要对每个 token 算 512 次点积。为了用上 VNNI/AMX 的 int8 算力，把 router 权重也量化到 int8。但量化会引入约 1% 的误差，这点误差可能让某个本该被选中的专家落选，导致最终输出算错。直接对 512 个专家全用 fp32 重算又太慢，等于白量化了。

解决办法是分两步走。第一步用 int8 快速筛出得分最高的 16 个候选专家，这一步快但可能有误差。第二步只对这 16 个候选用原始 fp32 权重重算一遍精确分数，再从中选 Top-K。这样 fp32 的计算量从 512 降到 16，但最终选择基于精确值。实测 S4 的输出和全 fp32 逐位一致，误差没进来。

粗筛阶段还能再省一步。筛 16 个候选只需要排序，不需要精确的 sigmoid 值，所以粗筛时直接用 logit 的线性近似代替 sigmoid（sigmoid 在 0 附近近似线性，斜率 0.25）。省掉 512×1024 次 sigmoid 计算，S4 快了 2%，输出逐位一致。

4.5 OpenMP 并行策略

前面几条主线里多次提到并行，这里集中讲一下 OpenMP 的任务划分和调度。

任务划分。实验给了三个粒度可选：按 token 切、按 expert 切、按投影矩阵的行切。大 N（S3/S4）按 token 切就行，128/1024 个 token 远超 16 线程，每个线程分到一批 token 互不干扰。小 N（S1/S2）只有 1 个 token、5 个专家任务，按 token 切只有 5 个任务，16 线程里 11 个空转。这时候退到更细的粒度：把单个 expert_ffn 内部的 gate/up/down 投影按 16 行一块切开，5 个专家 $\times (d_ff/16 + d_model/16)$ 块，够喂 16 线程了。粒度太细也不行（fork/barrier 开销超过计算），S1 的 $d_model=256$ 切出来

每块太小，实测反而更慢，所以 S1 保持 5 线程串行。

线程数。小 N 时把线程数 cap 到任务数，不硬开 16 个线程。5 个任务开 5 个线程，避免空闲线程的 fork 开销和 barrier 等待。

调度策略。per-task 和 intra 路径用 `schedule(static)`，每个任务工作量差不多，静态分配开销最低。batched FFN 路径用 `schedule(dynamic, 1)`，因为不同专家被选中的 token 数不一样，有的专家 8 个 token 有的是 0，静态分配会不均衡，动态调度让先干完的线程领下一批。

复用并行区域。intra-FFN 有 gate/up、requant、down 三个阶段，阶段之间有数据依赖（requant 要等 gate/up 算完才能重量化）。如果每个阶段单独开一个 `parallel for`，要 fork 三次。改成放进一个 `#pragma omp parallel` 里，用三个 `#pragma omp for` 衔接，阶段间的隐式 barrier 保证数据依赖，只 fork 一次。

4.6 其他关键修复

场景间隔离，场景内融合。这是后期最重要的修复，直接把 S1 从 10x 提到 13x、S2 从 16x 提到 19x。问题根因和前面 batched FFN 的一样：不同场景的代码挤在一个函数里，编译器把各场景的 intrinsics 互相内联，污染指令缓存。单独优化 S1 能到 12.4x，整合进主文件后掉到 10.5x，S2 也跟着回退。解决办法是每个场景拆成独立的 `noinline` 函数（`moe_forward_s1/s2/s3/s4`），阻断跨场景的 icache 污染；但场景内部的 router、量化、FFN、combine 不再用 `noinline` helper，而是内联在一起，让编译器跨阶段做寄存器复用和指令调度。即”场景间隔离，场景内融合”，S3/S4 不受影响。

xq stride bug: batched AMX 路径早期按 `d_model` 步幅索引 `xq_workspace`，但量化阶段写 token 用的是 `MAX_D_MODEL` 步幅，导致除首 token 外全错位。统一用 `MAX_D_MODEL` 修复。

intra-FFN (S2): S2 的 N=1 只有 5 个任务喂不饱 16 线程。把单个 `expert_ffn` 内部按 16 宽 f-block / d-block 拆成三阶段并行，扩展到 $5 \times (d_{ff}/16 + d_{model}/16)$ 个细任务。

4.7 优化时间线

下表按优化顺序汇总各步骤的场景与收益：

每一步都基于上一步的 VTune 热点定位：把 router 砍掉后 FFN 成首位，把 FFN 砍掉后 combine 成首位，把 combine 砍掉后 S4 热点落到 dpbusd 算力本身，接近指令级极限。S1 受 N=1 并行度天花板限制，加速比提升有限。

表 3: 优化时间线

阶段	优化	目标	收益
基线	VNNI 4× 展开 + 扁平并行 + AMX	全场景	基线建立
S4 router	batched AMX router	S4	router 33%→3.8%，82→87x
S4 FFN	batched FFN + 隔离	S4	87→151x
S3 FFN	放宽 batched 含 S3	S3	51→75x
combine	流水线融合	S3/S4	S3 -1.3%, S4 -0.5%
S1/S2	场景间隔离 + 场景内融合	S1/S2	S1 10→13x, S2 16→19x

5 性能数据与评分

5.1 四场景最终加速比

表 4: x86 主线最终成绩 (OJ judge, -c 16)

场景	baseline (s)	optimized (s)	加速比	得分
S1	1.033	0.0784	13.18x	56.24
S2	3.114	0.1604	19.42x	120.00
S3	1.326	0.0170	77.85x	75.01
S4	2.316	0.0150	154.80x	104.49
总分			89 / 120	

5.2 优化过程的热点演化 (S4)

下表展示 S4 在各优化阶段的热点占比演化，体现优化一个瓶颈后热点转移到下一个的过程：

表 5: S4 热点占比演化 (VTune, 1k iter --benchmark)

阶段	router	expert_ffn	dpbusd	首位瓶颈
基线	33%	12%	—	router
+AMX router	3.8%	38%	13s	expert_ffn load
+batched FFN	3.8%	20%	2.5s	combine
+combine 融合	3.8%	20%	2.5s	dpbusd

009 把 router 从 33% 砍到 3.8%；006 把 expert_ffn 从 13.84s 砍到 3.99s (-71%)、dpbusd 从 13s 砍到 2.5s (-81%)；combine 融合把 combine 从 5.82s 砍到约 3s 量级。S4 的热点最终落到 dpbusd 算力本身，已接近 VNNI + AMX 的指令级极限。

5.3 阶段耗时分解

表 6: 各场景阶段 wall 占比（插桩计时）

场景	router	FFN	combine	barrier
S1	9%	82%	9%	—
S2	1%	98%	1%	—
S3	9%	87%	3%	—
S4	3.8%	38%	3%	—

6 失败方向与教训

实验过程中试了不少失败的方向，总数达采用循环验证验证过的 idea 达 40+，下面是部分典型。

6.1 expert-major AMX batching（路由专家）

想法：主线一里 AMX 给 S4 的 router 提速很大，自然想把它也用到路由专家的 FFN 上。具体做法是先把所有 token 的 Top-K 结果扫一遍，按选中的专家 id 分桶，选中同一个专家的 token 聚到一起，然后用 AMX tile 一次算完这个专家对所有选中 token 的 FFN——相当于把 per-token GEMV 变成 expert-major 的 GEMM，让 AMX tile 的 16 行全部被选中 token 填满。

失败原因：FFN 的输出要累加到最终结果 y 上。分桶后不同专家的 token 列表互相重叠（同一个 token 可能被多个专家选中），多个线程往同一个 y 里写，得加锁防冲突，加锁的开销比 AMX 省的还大，S3 从 51x 掉到 21x。换成每个线程写自己的缓冲再归约也不行，归约时多个线程争着读同一个地址，算出来的结果全错了。结论是 AMX batch 路由专家这条路在当前负载下不如 per-token VNNI，但”每个任务写自己的缓冲、最后再合并”这个思路是对的，后面 batched FFN 就是用这个办法避开了写冲突。

6.2 NUMA interleave

想法：用 numactl --interleave=all 让数据均匀分布在两个 socket 的内存节点上，减少跨 socket 访存的延迟。

失败原因：实测几乎没效果（S3 +1%、S4 +1%，在噪声范围内）。查下来发现集群的 cgroup 把每个作业的 16 个 vCPU 钉在同一个 socket 上，内存也限在同一个 node，interleave 参数被截断成了空操作。这台集群根本没有跨 socket 的空间。

6.3 fork/env 调优

想法：调 OpenMP 的等待策略（OMP_WAIT_POLICY）和自旋参数（GOMP_SPINCOUNT），让 S1 每次 forward fork 线程组的开销更小。

失败原因：查了 libgomp 源码，默认的线程池会自旋约 3ms 才让出 CPU，而 S1 两次 forward 间隔只有 3us，线程始终在自旋不会真正睡眠。调参数（设成永久自旋或立即睡眠）都不改善，前者没区别，后者反而慢 60 倍。瓶颈不在 fork 本身，而是 5 个均匀任务只能让 5 个线程干活，这是数学上的上限，调度层面没法突破。

6.4 -ffast-math

想法：编译器加 -ffast-math 标志，让浮点运算（sigmoid、exp）的指令调度更激进，可能减少指令数。

失败原因：-ffast-math 会打乱浮点运算的精度和顺序，而 Top-K 选择对精度很敏感——两个专家的 sigmoid 分数差一点点，排序结果就可能反过来。自己写的 sigmoid 近似函数也会被干扰。算下来风险远大于收益，这个方向直接放弃。

7 实验总结与心得

这次实验最大的体会是优化得有方向，不能闷头试。每一步都是从 VTune 热点数据看出当前瓶颈在哪，再想对应的办法，而不是把各种技巧挨个试一遍。S4 从 router 到 FFN 到 combine，热点一步步往后挪，最后落到 dpbusd 算力本身，基本到极限了。S1 和 S3 的瓶颈都是 SwiGLU 三次 GEMV 的结构本身，数学上不可减少，优化空间有限。

另一个感受是优化方向主要得自己想，AI 帮不上太多。把同专家多 token 合并复用权重、router 换 batched AMX、combine 改单趟流式、场景间隔离场景内融合，这些结构性的改造都得从热点数据反推出来。但方向定了之后，实现代码、跑验证确实比人快。不过 AI 也会帮倒忙，估算收益经常偏乐观，二进制级副作用更是完全预测不到，最终还是得靠真实集群数据说话。

8 思考题

1.

以场景 $S3(N = 128, D = 256, H = 128, E = 16, K = 4)$ 为例, 估算参考实现一次前向的总访存量(专家权重被读了多少遍?)和总乘加次数, 计算算术强度($MACs/byte$)。按专家分组之后这两个数字分别变成多少? 由此说明这个负载是访存瓶颈还是计算瓶颈, 以及分组为什么能加速。

每个 token 激活 5 个专家(1 共享 + 4 路由), 每个专家有 gate、up、down 三个投影, 每个投影是 $D \times H$ 或 $H \times D$ 的矩阵。

乘加次数: 每个专家 $3 \times 256 \times 128 = 98304$ 次乘加, 5 个专家共 491520 次。加上 router 的 $16 \times 256 = 4096$ 次, 128 个 token 总共约 63.5M 次乘加。

访存量: 参考实现每个 token 把它选中的 5 个专家权重从头读一遍, 每个专家权重约 98304 字节。128 个 token 共读约 $128 \times 5 \times 98304 \approx 60MB$ 。但其中很多是重复读——同一个专家被多个 token 选中, 权重被读了又读。

算术强度: 乘加次数除以访存量, 约 $63.5M/60M \approx 1.06$ 次/字节。CPU 的 L2 带宽约 2TB/s、VNNI 算力约 1.7TMAC/s, 临界算术强度约 0.85 次/字节。当前 1.06 略高于临界, 算是轻度计算瓶颈, 但访存开销也不小。

按专家分组后: 如果先把任务按专家分组, 同一个专家的权重只读一次, 给多个 token 复用。S3 里每个专家平均被 32 个 token 选中, 访存量从 60MB 降到约 1.6MB, 算术强度从 1.06 升到约 40 次/字节。这远高于临界值, 瓶颈从访存变成了纯计算。分组能加速的根本原因就在这里: 算术强度提升约 40 倍, 权重不再是瓶颈。

2.

为什么激活用 *per-token scale*, 而权重每个矩阵一个 *scale* 就够了? 如果让一批 128 个 token 共享同一个激活 *scale*, 会发生什么?

激活是动态的, 每次输入的 token 值范围不一样。per-token scale 让每个 token 都能用满 int8 的范围 $[-127, 127]$, 精度损失最小。如果用全局 scale, 值小的 token 只能用到 int8 的一小截, 精度很差。权重是训练后固定的, 分布稳定, 一个矩阵一个 scale 就够用, 更细的粒度(per-row)能进一步降误差但反量化时更麻烦。

如果 128 个 token 共享同一个 scale: 取所有 token 的最大值作为 scale, 那么值小的 token 大部分元素都被量化到接近 0, 有效精度大幅下降, down 投影时误差会被放大。Top-K 选择可能还撑得住, 但 gate 值和 FFN 输出会明显跑偏。

3.

单个 $int8 \times int8$ 乘积的最大绝对值是多少? 为什么点积必须在 $int32$ 中累加——若改用 $int16$, 最坏情况下累加到第几项就会溢出? 归约长度在运行时变化, 请用允许

的最大归约长度 ($MAX_D_MODEL = 1024$) 估算最坏情况下的累加值, 并说明它离 $int32$ 的表示上限还有多少余量。

两个 $int8$ 相乘, 最大绝对值是 $127 \times 127 = 16129$ 。 $int16$ 最大值是 32767, 累加 3 项就可能溢出 (假设全同号且都取最大值)。

实际分布不会每项都取 127, 但归约长度可达 1024 (MAX_D_MODEL), 最坏情况下累加值为 $1024 \times 16129 \approx 16.5M$, 远超 $int16$ 上限 32767。 $int32$ 上限约 2.15×10^9 , 最坏累加值 16.5M 离 $int32$ 上限还有 $2.15 \times 10^9 / 16.5 \times 10^6 \approx 130$ 倍余量, 足够安全。所以点积必须在 $int32$ 里累加。